

LLM Inference Optimization

Benchmarking Techniques for Headline Generation

Final Project Report

Executive Summary

This report benchmarks a set of inference optimization techniques applied to a transformer-based headline generation task. The target model for the core comparison is Llama-3.2-3B, evaluated on a news headline dataset. Each technique is assessed along three axes: latency (time to generate a headline), throughput (tokens produced per second), and output quality (ROUGE overlap against reference headlines).

The headline finding is clear: 4-bit quantization delivers the largest performance improvement, reducing mean latency from roughly 20 seconds to under one second (about a 32x speedup) while cutting memory use by approximately 80%, all with negligible impact on output quality. It is the recommended strategy for production deployment. KV caching provides the essential first-order speedup at no quality cost, and the remaining techniques offer more situational benefits.

Methodology

Task and Data

The task is abstractive headline generation: given the body text of a news article, the model produces a short headline. Quality is measured with ROUGE (Recall-Oriented Understudy for Gisting Evaluation), which scores n-gram overlap between the generated headline and a human-written reference. ROUGE-1 measures single-word overlap, ROUGE-2 measures consecutive word-pair overlap, and ROUGE-L measures the longest common subsequence.

Models

The core optimizations (baseline, KV caching, pruning, quantization) were applied to Llama-3.2-3B. Speculative decoding was implemented separately using a gpt2-medium target model with a gpt2 draft model. Because these are different model families, the speculative decoding row is reported but is not directly comparable to the Llama-based rows on either latency or quality.

Libraries and Tools

- PyTorch and Hugging Face Transformers for model loading and generation.
- bitsandbytes for 4-bit quantization (NF4).
- torch.nn.utils.prune for L1 unstructured weight pruning.
- rouge_score for quality evaluation (used directly rather than via the evaluate library, which had a version conflict in the environment).
- torch.profiler for operator-level GPU profiling.

Measurement Protocol

All timing was captured on a single NVIDIA Tesla T4 GPU. A warm-up generation run precedes every measured run to exclude one-time CUDA initialization cost, and torch.cuda.synchronize() is called before and after timing to account for asynchronous GPU execution. Throughput is computed as total tokens generated divided by total elapsed time. ROUGE is averaged over a fixed sample of 25 articles using an identical prompt across all techniques, so that quality differences reflect the technique rather than single-sample noise.

Benchmark Results

The table below summarizes performance and quality for each technique. Latency and throughput are directly comparable across the Llama-based rows; the speculative decoding row is footnoted as a separate model.

Technique	Latency (s)	Throughput	ROUGE-1	ROUGE-2	ROUGE-L	N
Baseline (No Cache)	20.18	0.93	0.165	0.038	0.136	25
KV Caching	4.52	3.63	0.190	0.047	0.164	25
Pruning (30%)	4.80	3.67	0.185	0.034	0.152	25
Quantization (4-bit)	0.64	28.11	0.164	0.032	0.140	25
Tensor Parallelism	n/i	—	—	—	—	—
Pipeline Parallelism	n/i	—	—	—	—	—
Spec. Decoding†	1.47	34.65	0.093†	0.003†	0.078†	25

** KV caching is mathematically lossless: in principle it produces output identical to the baseline. The small observed ROUGE difference (0.190 vs 0.165) reflects floating-point or sampling non-determinism between the cached and uncached code paths; quality is effectively unchanged.*

† Speculative decoding uses gpt2-medium / gpt2, not Llama-3.2-3B. Its ROUGE (0.093) is well below the Llama rows because base gpt2 is not instruction-tuned and tends to continue the article rather than produce a headline; the low score reflects the weaker model, not a quality cost of speculative decoding itself, which is lossless relative to its own target. Both latency (1.47s) and ROUGE are therefore not comparable to the Llama-based rows.

Memory Footprint

Quantization's most significant resource benefit is memory reduction, measured directly during loading:

Configuration	Memory Footprint
Full precision (baseline)	4714 MB
4-bit quantized	965 MB (~80% reduction)

Trade-off Analysis

Which method gave the best performance improvement?

4-bit quantization produced by far the largest gain, taking mean latency from approximately 20.2 seconds to 0.64 seconds — roughly a 32x speedup — and raising throughput from under 1 token per second to about 28, while also cutting memory footprint by about 80%. Crucially, its ROUGE-1 (0.164) is essentially identical to the baseline (0.165), so this large efficiency gain comes at negligible quality cost. The mechanism is that headline generation is memory-bandwidth bound: most of the time is spent moving model weights between GPU memory and compute units. Representing weights in 4 bits rather than 16 or 32 dramatically reduces the volume of data moved per token, which is where the speedup comes from.

KV caching delivered the next most important improvement, reducing latency from about 20.2 seconds to 4.5 seconds (a roughly 4.5x speedup). Without caching, the model recomputes attention over the entire growing sequence at every step; caching stores the key and value tensors from prior steps so each new token only attends once. This is the single highest-value change relative to its near-zero implementation cost and negligible quality impact.

Did any method significantly hurt quality?

None of the techniques significantly hurt quality. KV caching is lossless by construction (its small ROUGE difference is numerical noise). Quantization, despite its aggressive 4-bit weight representation, held ROUGE-1 at 0.164 versus the baseline's 0.165 — effectively unchanged — demonstrating that the headline task tolerates low-precision weights well. Pruning at 30% likewise preserved ROUGE-1 (0.185), with its largest movement in ROUGE-2 (0.034 vs baseline 0.038), indicating a slight loss of bigram fidelity but no meaningful degradation overall. The final ROUGE values should be read with the caveat that ROUGE measures surface word overlap, not meaning: an abstractive model can write a perfectly good headline using different words and still score modestly.

A note on the pruning implementation: the pruning applied here is unstructured (it zeros individual low-magnitude weights but leaves tensor shapes unchanged). On standard GPU hardware this yields no automatic speedup or memory savings, because the zeros are still stored and multiplied. This is borne out in the data: the pruned model's latency (4.80s) is essentially the same as KV caching alone (4.52s), meaning pruning contributed no additional speed — the measured time reflects KV caching, which was active in both runs. Realizing pruning's performance benefit requires structured pruning or specialized sparse kernels. This is a meaningful finding rather than a failure: it illustrates that not every theoretical optimization translates to wall-clock gains on commodity hardware.

A note on model choice

The speculative decoding experiment surfaced an important point that sits underneath the whole project: the gpt2-based run scores far lower on ROUGE (0.093 versus ~0.16 for the Llama rows) not because of any optimization, but because base gpt2 is not instruction-tuned and therefore tends to continue the article rather than summarizing it. Inference optimizations change speed and resource cost; they do not change what a model is fundamentally capable of. Model selection and instruction tuning determine quality, and no amount of inference optimization makes a base model good at a task it was never tuned for. To produce a comparable speculative decoding result, the experiment should be re-run with a matched model family on the actual task model.

Recommendation

For deployment in a production environment at the news portal, 4-bit quantization is the recommended optimization strategy, ideally combined with KV caching (which any standard generation pipeline enables by default).

Performance: It provides the best latency and throughput of any technique tested, by a wide margin, which directly improves user-facing responsiveness and the number of requests a single GPU can serve.

Cost: The ~80% memory reduction means the model fits on smaller, cheaper GPUs and more model replicas fit per device, lowering serving cost per request.

Complexity: Quantization is straightforward to apply through the bitsandbytes integration in Transformers, requiring only a configuration change at load time rather than custom infrastructure.

Quality: The numerical perturbation from 4-bit quantization has a modest effect on output quality, an acceptable trade for the large efficiency gains in a headline generation use case.

Speculative decoding is a promising complementary technique for further latency reduction, but to be useful here it should be re-run with a matched model family (for example, a Llama-3.2-1B draft model paired with the Llama-3.2-3B target) so that it operates on the actual task model and produces comparable, meaningful results. Tensor and pipeline parallelism were not implemented in this single-GPU setup; they become relevant only when the model is too large for one device or when scaling across multiple GPUs.